

Линковка в C++

Сениченков Пётр

2026

Линковка — это этап сборки, на котором отдельные единицы трансляции (*translation unit*), единицы подстановки шаблонов (*instantiation unit*) и внешние библиотеки собираются в исполняемый файл или библиотеку (*program image*).

Линковка — это этап сборки, на котором отдельные единицы трансляции (*translation unit*), единицы подстановки шаблонов (*instantiation unit*) и внешние библиотеки собираются в исполняемый файл или библиотеку (*program image*).

All external entity references are resolved. Library components are linked to satisfy external references to entities not defined in the current translation. All such translator output is collected into a program image which contains information needed for execution in its execution environment.

ISO/IEC 14882:2020, 5.2, translation phase 9

Немного терминологии

Чтобы не запутаться, будем использовать не совсем стандартные термины (если не очевидно, о чём идёт речь):

- Compile-time
 - **Компоновка**
 - 9 фаза трансляции
 - Link-editor
- Runtime
 - **Системный интерпретатор**
 - ld.so
 - Runtime linker
 - Dynamic linker/loader

Что делает линкер

- Разрешение символов
- Объединение секций
- Address relocation
- Подключение динамических библиотек

Что делает линкер

- Разрешение символов
- Объединение секций
- Address relocation
- Подключение динамических библиотек
 - Position independent code (PIC)
 - Compile-time
 - Создаются Global offset table (GOT) и Procedure linkage table (PLT)
 - Заполняются динамические секции ELF
 - Runtime
 - Запускается `ld.so <executable>`
 - Находит библиотеки из `DT_NEEDED`, обходит их рекурсивно и загружает
 - Заполняет адреса в GOT
 - При первом вызове заполняется PLT

Storage duration, linkage, visibility

- Storage duration
 - Static
 - Thread
 - Automatic
 - Dynamic
- Linkage
 - No linkage
 - Internal
 - External
 - Language
 - Module
- Visibility

Name mangling, language linkage

- Name mangling
- Language linkage
 - `extern "C++"`
 - `extern "C"`
 - Отключает name mangling
 - Реализации могут поддерживать другие языки
 - На практике это не используется

Name mangling, language linkage

```
int foo(double, long);           // → _Z3foodl
extern "C" int bar(double, long); // → bar

extern "C" typedef void FUNC();
FUNC* baz;                       // → baz
extern "C" FUNC* qux;            // → qux

extern "C" {
    struct C {
        void mem(void (*)());    // → _ZN1C3memEPFvvE
    };
}

struct D {
    void mem(void (*)());        // → _ZN1D3memEPFvvE
};
```

- **Не больше одного** определения в одной translation unit
- **Ровно одно** определение для каждой не-inline функции или inline-переменной, если она используется
- **Полное** определение для каждого класса, если он используется
- Для класса, inline-функции, inline-переменной, перечисления, шаблона (но не полной специализации) разрешено несколько определений, если они *полностью совпадают* и определены в разных translation unit (модулях)

- Copy relocation
 - Сам исполняемый файл обычно создаётся из position-dependent кода. Нельзя использовать GOT и PLT для разрешения адресов
 - Когда компоновщик видит ссылку на данные, определённые в разделяемом объекте, он создаёт пустую запись в .bss, того же размера, что и эти данные
 - Интерпретатор копирует данные в эти записи

- Itanium C++ ABI¹

¹<https://itanium-cxx-abi.github.io/cxx-abi/abi.html>

- Borland model

- Компилятор подставляет шаблоны там, где они используются
- Линкер “склеивает” одинаковые подстановки
- Нужно подменять линкер
- Дольше компиляция

- Borland model

- Компилятор подставляет шаблоны там, где они используются
- Линкер “склеивает” одинаковые подстановки
- Нужно подменять линкер
- Дольше компиляция

- Cfront model

- Глобальный реестр подстановок
- Можно использовать системный линкер
- Гораздо сложнее в реализации

Виртуальные функции

- *TODO: виртуальные функции в контексте линковки*
- vtable размещается в объектном файле, который содержит *key function*
Key function is defined as the first non-inline, non-pure, virtual function declared in the class. If you do not provide a definition for the key function (or fail to link to the file providing the definition) then the linker will not be able to find the class' vtable.
- Если **не объявлено** ни одной не-inline, не чистой, виртуальной функции, то компилятор сгенерирует vtable в каждой TU, которая ссылается на этот класс (в секции COMDAT)

- Компилятор записывает промежуточное представление (GIMPLE, LLVM bitcode) в объектные файлы
- Link-time optimizer читает промежуточное представление и работает вместе с линкером
 - Линкер подключает `liblto_plugin.so` или `libLTO.so`

- Компилятор записывает промежуточное представление (GIMPLE, LLVM bitcode) в объектные файлы
- Link-time optimizer читает промежуточное представление и работает вместе с линкером
 - Линкер подключает `liblto_plugin.so` или `libLTO.so`
- В основном, те же оптимизации, что и при компиляции, но оптимизатор видит несколько TU:
 - Cross-module inlining
 - Dead code elimination
 - Devirtualization
 - Identical code folding
 - Code layout optimization
 - COMDAT folding

- Единица компиляции — crate
- Экспорт символов через модули, а не текстовые подстановки
- Name mangling
 - Содержит имя и хэш crate
- Нет стабильного ABI
- rlib содержит дополнительные метаданные
 - Версия компилятора
 - Crate id
 - Информация об исходных файлах
 - Информация об экспортированных символах
 - Mid-level IR
 - LLVM bitcode
- Generic monomorphization

Rust foreign function interface

- C ABI
- unsafe
 - Линковка в runtime
 - Коллизии имён без mangling

```
#[link(name = "my_c_library")]
unsafe extern "C" {
    fn my_c_function(x: i32) -> bool;
}
```

```
#[unsafe(no_mangle)]
pub extern "C" fn callable_from_c(x: i32) -> bool {
    x % 3 == 0
}
```

- Единица сборки — package
 - Export data и метаданные
- В основном используется внутренний линкер
- Self-contained binary
 - Динамические библиотеки практически не используются
- Относительно стабильный ABI

Inter-language linking in Go

```
package pkg

// #cgo LDFLAGS: -lpthread
// #include <stdio.h>
// void myprintf(char* s) { printf("%s\n", s); }

import "C"
import "unsafe"

//export MyFunction
func MyFunction(arg1, arg2 int) int64 {...}

func main() {
    cs := C.CString("Hellow from stdio")
    C.myprint(cs)
    C.free(unsafe.Pointer(cs))
}
```

Как это выглядит в коде

- Псевдо-пакет "C"
- Комментарий перед `import "C"` содержит
 - Флаги для компиляторов (`// #cgo <environmental variable>`)
 - Общий заголовок для сгенерированного кода на C
- Экспорт функций с помощью комментария `//export <name>`
- Обращение к переменным и функциям C через `C.<name>`
- Не все возможности C поддерживаются
 - Variadic функции
 - Вызов функции по указателю
- Нужно постоянно приводить типы
- Хрупкая работа с указателями

Как это работает

- Вызывается не компилятор, а `go tool cgo`
- Собирает все исходники C, C++ и Fortran в текущей директории
 - Но не в поддиректориях
- Генерирует несколько файлов на C и Go и собирает их
 - Экспортируемые файлы на C для подключения из внешних программ
 - Обёртки над использованными функциями на C
 - Исходный файл на Go с подставленными типами и именами из псевдо-модуля "C"
 - Вспомогательные файлы с обёртками над типами, некоторыми статическими проверками, etc.
 - Подробнее — в репозитории с примерами

Системы конфигурации и сборки для C++

- Системы сборки
 - Просто запускают команды
 - topsort
- Системы конфигурации
 - Мало знают о структуре программы

Системы конфигурации и сборки для C++

- Системы сборки
 - Просто запускают команды
 - topsort
- Системы конфигурации
 - Мало знают о структуре программы

```
// main.cpp  
#include "lib.h"
```

```
# CMakeLists.txt  
add_executable(MyProgram main.cpp)  
target_link_libraries(MyProgram MyLibrary)
```

Системы сборки для Rust и go

- Cargo
 - Package manager
 - Build orchestrator
 - Структура проекта стандартизирована
 - Cargo.toml
 - src/main.rs или src/lib.rs
- Go toolchain
 - Единый toolchain, тесно интегрированный в сам язык
 - Build graph $\leftrightarrow$ import graph
 - Разработчик практически не контролирует сборку